Student Assessment Submission and Declaration

When submitting evidence for assessment, you must sign a declaration confirming that the work is your own.

Student name:	Pawel Mateusz Czerepski	ESL ID No	120071
		BSU ID No	575378
Submission date:			09.10.2026
Programme:	Computing		
Module name and code:	Code lab 2		
Title:	Assessment 2 Data driven app		
Assessor name:	Sohail Ilyas		

Plagiarism

Plagiarism is a form of cheating. Plagiarism must be avoided at all costs and students who break the rules, however innocently, may be penalised. It is your responsibility to ensure that you understand correct referencing practices. As a university level student, you are expected to use appropriate references throughout and keep carefully detailed notes of all your sources of materials for material you have used in your work, including any material downloaded from the Internet. Please consult the relevant unit lecturer or your course tutor if you need any further advice.

Student declaration I certify that the assignment submission is entirely my own work. I fully understand the consequences of plagiarism. I understand that making a false declaration is a form of malpractice.			
Student signature:	Pawel Mateusz Czerepski	Date:	09.10.2026

Contents

Student Assessment Submission and Declaration 1
DEVELOPMENT DOCUMENT 2
Introduction: 2
Link to repository: 2
Serwer Code 4
Client code 4

DEVELOPMENT DOCUMENT

Introduction:

Ratewise is a one-page platform for desktop or phone users with intentions to estimate currencies on conversion. The user inputs an amount and views the rates and converted value after choosing a sending or receiving currency. The currency converter offers a copy-result action, a list of common destinations, and currency swapping. Daily updates for the open-access service on ExchangeRate-API activates an Express endpoint that retrieves exchange rates. The front end is structured in a way that it can easily transition from the mobile layout to the desktop layout.

Link to repository:

Project Plan:

The first Vite display was not a converter, but just a framework starter, which I connected to the Express endpoint after replacing it with the rate panel and conversion form. While implementing it, I made adjustments to the client for old rates clearance in the instance of loading a new base currency. This prevents the presentation of stale results as existing new processes. Validating the platform included successfully requesting USD rates, a mobile viewport check, and USD and EUR swapping. The production and Vite lint passed (Vite, 2026).

Activity	Week 1	Week 2	Week 3	Week 4	Week 5
Framework Starter Setup (Vite)	**X**				
Express Endpoint Connection		**X**			
Rate Panel & Form Integration			**X**		
Stale Rates Clearance & Fixes				**X**	

Validation & Production Linting	**X**

Design:

The target-currency selector, base-currency selector, and amount field controlled React inputs. The corresponding rate in the selection and the chosen amount calculate the target value. Destinations are directly selected from common-currency rows. The interface has separate displays for short-indicative rate caveat, provider's update date, total, and the exchange rates.

The interface applies a secondary rate-details panel, grouped conversion fields, clear heading hierarchy, and a restrained palette. A cool live region announces the conversion result, there is an accessible name on the swap control, and currency controls retain visible labels. Also, there is a visible outline on the keyboard focus. The aspects maintain the prominence of reading the results and entering an amount among other primary tasks, and retain the rate context with no instance of overcrowding.

There is a separation between the data request and the application's interface. The server uses Express, while the client uses Vite and React. On the client side, the requested rate set is defined by the selected base currency. The removal of the component or a quick change in the base selection causes the absolute request to be cancelled by an AbortController. The interface does not indicate a fabricated conversation, but an error in case a request fails. Also, it shows a status indicator when rates load.

Testing:

The first Vite display was not a converter, but just a framework starter, which I connected to the Express endpoint after replacing it with the rate panel and conversion form. While implementing it, I made adjustments to the client for old rates clearance in the instance of loading a new base currency. This prevents the presentation of stale results as existing new processes. Validating the platform included successfully requesting USD rates, a mobile viewport check, and USD and EUR swapping. The production and Vite lint passed (Vite, 2026).

Test Case	Test Data	Expected Results	Actual Results	Pass / Fail
TC01: Express Endpoint Validation	Request USD rates from API	Successful retrieval of active USD data sets	USD rates requested successfully with no failures	**Pass**
TC02: Viewport Responsiveness	Narrow width / Mobile width	Interface transitions easily without overcrowding	Responsive check passed layout scaled cleanly	**Pass**

TC03: Currency Control Swapping	Swap control selection (USD/EUR)	Base and target switch positions and values	USD and EUR swapped with visible outlines intact	**Pass**
TC04: Asynchronous Timing Check	Rapid base selection change	Clear old rates instance to prevent stale processes	Old rates cleared, no presentation of stale results	**Pass**

Critical Reflection:

The direct nature of the conversion task is the key strength of this version. As the provider attribution, update date, and rate define the result, there is a consistent visibility for both currencies and amount. Desktop and narrow widths were used to check the responsive layout, while the API dependency was made apparent by loading and error states.

One limitation in the implementation is the loss of server cache when the platform restarts, for it is in memory. If excessively large, negative, and empty amounts are handled clearly, it can strengthen input usability. The most challenging part of React and Express was during the asynchronous request, for the bugs are sometimes timing-dependent and invisible.

AI Acknowledgement:

- **Acknowledgement:** I used a bit of GitHub Copilot (2026) AI assistance in styling the converter and drafting the Express rate endpoint.

References:

- ExchangeRate-API (2026) "Open Access Endpoint". exchangerate-api.com.
- Vite (2026) "Getting Started". vite.dev.

Appendix:

Serwer

```
const express = require('express');

const cors = require('cors');


const app = express();

const PORT = 3001;
```

```
// Enable Cross-Origin Resource Sharing for secure client-server communication
app.use(cors());
app.use(express.json());

// Set live access token string for freecurrencyapi setup configurations
fca_live_NHGFjKK3xDx02uCtGEzkuWWrOWAnpaCvOZJe6QtU

// Core server endpoint requested by the React client application layer
app.get('/api/rates', async (req, res) => {
    // Read targeted base currency argument, fallback default to USD
    const baseCurrency = req.query.base || 'USD';

    try {
        // Query official freecurrencyapi latest currency matrix stream
        const apiUrl =
`https://freecurrencyapi.com{API_KEY}&base_currency=${baseCurrency}`;

        const response = await fetch(apiUrl);

        if (!response.ok) {
            throw new Error(`API response status: ${response.status}`);
        }

        const json = await response.json();

        // Standardize raw response structure into uniform data layout
        res.json({
```

```
            base: baseCurrency,
            rates: json.data,
            date: new Date().toISOString().split('T')[0]
        });

    } catch (error) {
        console.error("Error fetching currency rates:", error);
        // Retain native application catch loop state tracking structure
        res.status(500).json({ error: "Failed to retrieve exchange rates from freecurrencyapi."
});
    }
});

app.listen(PORT, () => {
    console.log(`Currency API listening on http://localhost:${PORT}`);
});
```

Client

```
import { useEffect, useMemo, useState } from 'react'
import './App.css'

const displayNames = new Intl.DisplayNames(['en'], { type: 'currency' })
const popularCodes = ['EUR', 'GBP', 'JPY', 'CAD', 'AUD', 'CHF']
const currentYear = new Date().getFullYear()

function currencyName(code) {
  try {
    return displayNames.of(code) || code
```

```
  } catch {
    return code
  }
}

function formatMoney(value, currency, maximumFractionDigits = 2) {
  try {
    return new Intl.NumberFormat('en', {
      style: 'currency',
      currency,
      minimumFractionDigits: 2,
      maximumFractionDigits,
    }).format(value)
  } catch {
    return `${value.toFixed(2)} ${currency}`
  }
}

function App() {
  const [base, setBase] = useState('USD')
  const [target, setTarget] = useState('EUR')
  const [amount, setAmount] = useState('1')
  const [rates, setRates] = useState(null)
  const [updatedAt, setUpdatedAt] = useState('')
  const [loading, setLoading] = useState(true)
  const [error, setError] = useState('')
  const [copied, setCopied] = useState(false)

  useEffect(() => {
```

```
    const controller = new AbortController()

    async function loadRates() {
      setLoading(true)
      setRates(null)
      setUpdatedAt('')
      setError('')
      try {
        const response = await fetch(`/api/rates/${base}`, { signal: controller.signal })
        const data = await response.json()
        if (!response.ok) throw new Error(data.error || 'Rates could not be loaded.')
        setRates(data.rates)
        setUpdatedAt(data.updatedAt)
      } catch (requestError) {
        if (requestError.name !== 'AbortError') {
          setError(requestError.message || 'Rates could not be loaded. Check that the API server is running.')
        }
      } finally {
        if (!controller.signal.aborted) setLoading(false)
      }
    }

    loadRates()
    return () => controller.abort()
  }, [base])

  const currencyCodes = useMemo(() => Object.keys(rates || {}).sort(), [rates])
  const numericAmount = Number.parseFloat(amount) || 0
```

```
  const rate = rates?.[target]

  const converted = rate == null ? null : numericAmount * rate

  const rateLabel = rate == null ? '—' : new Intl.NumberFormat('en', { maximumSignificantDigits: 8 }).format(rate)

  const dateLabel = updatedAt ? new Date(updatedAt).toLocaleDateString('en', { day: 'numeric', month: 'short', year: 'numeric' }) : ''


  function swapCurrencies() {

    setBase(target)

    setTarget(base)

  }


  async function copyResult() {

    if (converted == null) return

    try {

      await navigator.clipboard.writeText(`${formatMoney(converted, target, 4)} (${amount || '0'} ${base} → ${target})`)

      setCopied(true)

      window.setTimeout(() => setCopied(false), 1800)

    } catch {

      setError('Clipboard access is not available in this browser.')

    }

  }


  return (

    <div className="app-shell">

      <header className="topbar">

        <a className="brand" href="#top" aria-label="Ratewise home">

          <span className="brand-mark" aria-hidden="true">↗</span>

          <span>ratewise<span className="brand-period">.</span></span>
```

```
      </a>
      <div className="topbar-note"><span className="live-dot" /> Rates updated daily</div>
    </header>

    <main id="top">
      <section className="intro">
        <div className="eyebrow"><span>✳</span> SIMPLE, CLEAR CURRENCY CONVERSION</div>
        <h1>Money moves.<br /><span>Know the rate.</span></h1>
        <p className="intro-copy">Convert currencies with the latest available exchange rates. No guesswork, just a clear number.</p>
      </section>

      <div className="workspace">
        <section className="converter-card" aria-labelledby="converter-heading">
          <div className="card-heading">
            <div>
              <p className="card-kicker">CONVERTER</p>
              <h2 id="converter-heading">Make a conversion</h2>
            </div>
            <span className="secure-mark" aria-label="Exchange rates">⇄</span>
          </div>

          <div className="currency-fields">
            <div className="currency-field">
              <label className="field-label" htmlFor="amount-input">You send</label>
              <span className="input-row">
                <input
                  id="amount-input"
```

```
            type="number"
            min="0"
            step="any"
            inputMode="decimal"
            value={amount}
            onChange={(event) => setAmount(event.target.value)}
            aria-label={`Amount in ${base}`}
          />
          <span className="field-divider" />
          <select value={base} onChange={(event) => setBase(event.target.value)} aria-label="From currency">
            {currencyCodes.length ? currencyCodes.map((code) => <option key={code} value={code}>{code}</option>) : <option value={base}>{base}</option>}
          </select>
        </span>
        <span className="currency-name">{currencyName(base)}</span>
      </div>

      <button className="swap-button" type="button" onClick={swapCurrencies} aria-label="Swap currencies" title="Swap currencies">
        <span aria-hidden="true">⇅</span>
      </button>

      <div className="currency-field">
        <span className="field-label">You receive</span>
        <span className="input-row target-row">
          <span className="result-amount" aria-live="polite">{converted == null ? '—' : formatMoney(converted, target, 4)}</span>
          <span className="field-divider" />
          <select id="target-currency" value={target} onChange={(event) => setTarget(event.target.value)} aria-label="To currency">
```

```
                {currencyCodes.length ? currencyCodes.map((code) => <option key={code} value={code}>{code}</option>) : <option value={target}>{target}</option>}

              </select>

            </span>

            <span className="currency-name">{currencyName(target)}</span>

          </div>

        </div>


        <div className="rate-summary">

          <span className="rate-icon" aria-hidden="true">↗</span>

          <span>1 {base} <span className="muted-arrow">=</span> <strong>{rateLabel} {target}</strong></span>

          <button className="copy-button" type="button" onClick={copyResult} disabled={converted == null}>

            {copied ? 'Copied ✓' : 'Copy result'}

          </button>

        </div>


        {error && <div className="error-message" role="alert">{error} <span>Make sure the Express server is running.</span></div>}

        {loading && <div className="loading-line" role="status"><span className="spinner" /> Updating rates…</div>}

        {!loading && !error && updatedAt && <p className="updated-note">Rate data last updated {dateLabel} · indicative rates only</p>}

      </section>


      <aside className="side-card" aria-label="Rate details">

        <div className="side-card-top">

          <div>

            <p className="card-kicker">AT A GLANCE</p>

            <h2>Today’s rate</h2>
```

```
          </div>
          <span className="trend-mark" aria-hidden="true">↘</span>
        </div>
        <div className="big-rate">{rateLabel}<span> {target}</span></div>
        <p className="rate-caption">for every 1 {base}</p>
        <div className="side-divider" />
        <div className="popular-heading"><span>Popular rates</span><span>1 {base}</span></div>
        <div className="popular-list">
          {popularCodes.filter((code) => code !== base && rates?.[code] != null).map((code) => (
            <button className={`popular-row ${target === code ? 'selected' : ''}`} key={code} type="button" onClick={() => setTarget(code)}>
              <span className="popular-currency"><span className="currency-token">{code.slice(0, 1)}</span><span><strong>{code}</strong><small>{currencyName(code)}</small></span></span>
              <strong className="popular-value">{new Intl.NumberFormat('en', { maximumSignificantDigits: 6 }).format(rates[code])}<span> ↗</span></strong>
            </button>
          ))}
          {!rates && <p className="popular-placeholder">Popular currencies will appear when rates load.</p>}
        </div>
        {updatedAt && <p className="side-updated">Last update · {new Date(updatedAt).toLocaleTimeString('en', { hour: '2-digit', minute: '2-digit', timeZone: 'UTC', timeZoneName: 'short' })}</p>}
      </aside>
    </div>

    <section className="trust-strip" aria-label="Rate information">
```

```
        <div className="trust-item"><span className="trust-icon">◷</span><span><strong>Daily rate updates</strong><small>Fresh data from the rates provider</small></span></div>

        <div className="trust-item"><span className="trust-icon">◎</span><span><strong>Clear before you convert</strong><small>See the rate and result up front</small></span></div>

        <div className="trust-item"><span className="trust-icon">◇</span><span><strong>For quick estimates</strong><small>Not intended for financial decisions</small></span></div>

      </section>

    </main>


    <footer className="footer">

      <span>© {currentYear} ratewise · Made for everyday conversions</span>

      <a href="https://www.exchangerate-api.com" target="_blank" rel="noreferrer">Rates By Exchange Rate API <span aria-hidden="true">↗</span></a>

    </footer>

  </div>

 )

}


export default App
```